

Get-IndexedItem

Query the Windows Search Index instead of fighting it

If you've ever run `Get-ChildItem -Recurse` or File Explorer's search bar against a drive with hundreds of thousands of files, you know the pain: minutes of spinning disks, CPU spikes, and a search that walks every folder from scratch — every single time.

Windows has already solved this problem quietly, in the background, for years: the **Windows Search Index (WSI)**, a database that catalogs file names, extensions, metadata, and — for supported formats — full file content. `Get-IndexedItem` is a PowerShell module, that queries that database directly, instead of asking PowerShell to walk the filesystem live. Same data, radically faster access.

Why It's Fast

`Get-ChildItem` and `Get-IndexedItem` don't do the same job under the hood:

- `Get-ChildItem` opens folders one at a time and reads their contents live.
- `Get-IndexedItem` sends a SQL-style query directly to the Windows Search Index database (via the Search.CollatorDSO OLE DB provider), which has already done the expensive work of indexing names, properties, and text content ahead of time.

You're not searching the drive. You're searching a pre-built catalog of it.

Installing

```
Install-Module Get-IndexedItem -Force
Import-Module Get-IndexedItem -Force
```

!NOTE

`Get-IndexedItem` can only return what the Windows Search Index has already indexed. By default, not everything on your machine is covered. To widen its reach: open Control Panel → search “Indexing Options” → click Modify and select the drives/folders you want indexed → give Windows time to finish building the index before querying large volumes.

Syntax, In Brief

The core parameter is `-Filter` (alias `-Include`), which accepts one or more strings that behave like a SQL WHERE clause:

- Multiple `-Filter` strings are automatically joined with **AND**.
- A single `-Filter` string can contain **AND** / **OR** itself.
- A bare term with no operator (`=`, `>`, `<`, **LIKE**, **CONTAINS**) is treated as a free-text search.
- `-Path` scopes the search to a folder (add `-Recurse` for subfolders); omit it to search the whole index.

- -OrderBy (alias -Sort), -First (alias -Top), and -Value (alias -Group) round out the everyday toolkit.

Examples

1. Free-text search across everything indexed

```
Get-IndexedItem -Filter "Password"
```

Returns every indexed item — filename, content, or metadata — containing "password."

2. Restrict to a document type

```
Get-IndexedItem -Filter "password", "kind='document'"
```

Two filter strings, auto-joined with AND: only documents (.docx, .pdf, .pptx, .xlsx, etc.) that contain "password."

3. Wildcard filename match

```
Get-IndexedItem -Filter "filename='%cyber%'"
```

% is the AQS wildcard character — matches any filename containing "cyber," similar to *cyber* elsewhere.

4. Explicit AND inside a single filter string

```
Get-IndexedItem -Filter "System.Kind = 'Music' AND AlbumArtist LIKE '%'"
```

One string, one WHERE clause, two conditions — functionally identical to passing them as two separate -Filter strings.

5. OR across values of the same field

```
Get-IndexedItem -Filter "kind = 'document' OR kind = 'email'"
```

Returns both documents and emails matching the rest of the query.

NOTE

Mixing AND and OR with parentheses in one string (e.g. "Kind='Music' AND (Rating='4' OR Rating='5')") is valid SQL and sometimes works, but has been reported by some users to error out depending on module version and search provider. If a combined query fails, split it into two simpler -Filter calls and merge the output with Sort-Object -Unique instead of fighting the parentheses.

6. Date comparison

```
Get-IndexedItem -Filter "kind=music", "DateModified>'2012-05-31'" -NoFiles |  
Select-Object -ExpandProperty Name
```

Files modified after a given date. Dates use year-month-day format; since the underlying field is a full datetime, an exact = only matches items saved at midnight.

7. Free-text CONTAINS() combined with metadata filters

```
Get-IndexedItem -Filter "Contains(*,'confidential')", "kind='picture'", "keywords='portfolio'" `
-Path ~ -Recurse
```

Pictures tagged "portfolio" that also contain "confidential" anywhere in their indexed properties.

8. Sort and limit results

```
Get-IndexedItem -Filter "kind='document'" -OrderBy "System.Size DESC" -First 10
```

The 10 largest indexed documents.

9. Discover the distinct values of a field

```
Get-IndexedItem -Value "kind" -Recurse
```

Lists every distinct "kind" present in the index — handy for finding the exact value to use before writing a precise filter.

10. Search a remote, indexed network share

```
Get-IndexedItem -Filter "kind=recordedtv" -Path "\\filesrv\shared" -Recurse
```

Same query engine, pointed at a share instead of the local disk.

11. Pipe results straight into an action

```
Get-IndexedItem -Filter "kind='document'", "password" -Recurse |
Copy-Item -Destination D:\Review
```

Query, then act — no intermediate variable needed.

A Security Angle Worth Knowing

Here's what makes this cmdlet interesting from a security standpoint, not just a productivity one: the same script also ships inside **Empire**, a well-known open-source post-exploitation framework, as a data-discovery (“collection”) module. The use case there mirrors Example 7 almost exactly — instead of crawling a compromised host's filesystem folder by folder (slow, and noisy on disk I/O), an operator queries the index Windows already built and gets files containing keywords like “password” or “confidential” back in seconds.

It's a clean, real-world example of “living off the land”: a completely legitimate PowerShell module, built entirely on a built-in Windows service, doing exactly what it was designed to do — just aimed at reconnaissance instead of convenience. For defenders, it's a reminder that search and indexing infrastructure deserves a place in detection engineering, not because the tool itself is malicious, but because its usage pattern can be a signal worth watching for.

Takeaway

Get-IndexedItem illustrates a broader lesson: Windows already tracks an enormous amount of metadata about your system, and most of it is queryable if you know where to look. Whether you're speeding up your own file management or thinking about what an indexed search reveals from an attacker's perspective, understanding the tool beats searching blind.